



Aislamiento y límites de confianza en producción

Diseñando sistemas que contienen el impacto de los fallos

[Jorge de la Flor](#) | Software & Cyber-Physical Systems Developer

CS Week Perú 2026

La Pregunta Fundamental

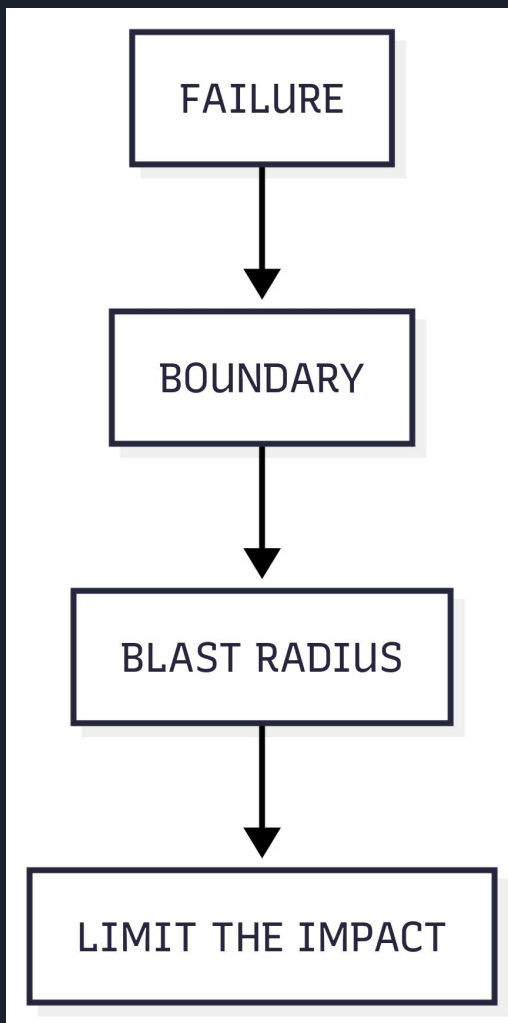
"¿Qué significa realmente que un sistema sea estable en producción?"

Prevenir el fallo es la primera línea de defensa. Pero un sistema robusto también debe definir qué ocurre cuando un fallo sucede.

Blast Radius

¿Hasta dónde permites que se propague el fallo?

Limitar el radio de impacto de los fallos.

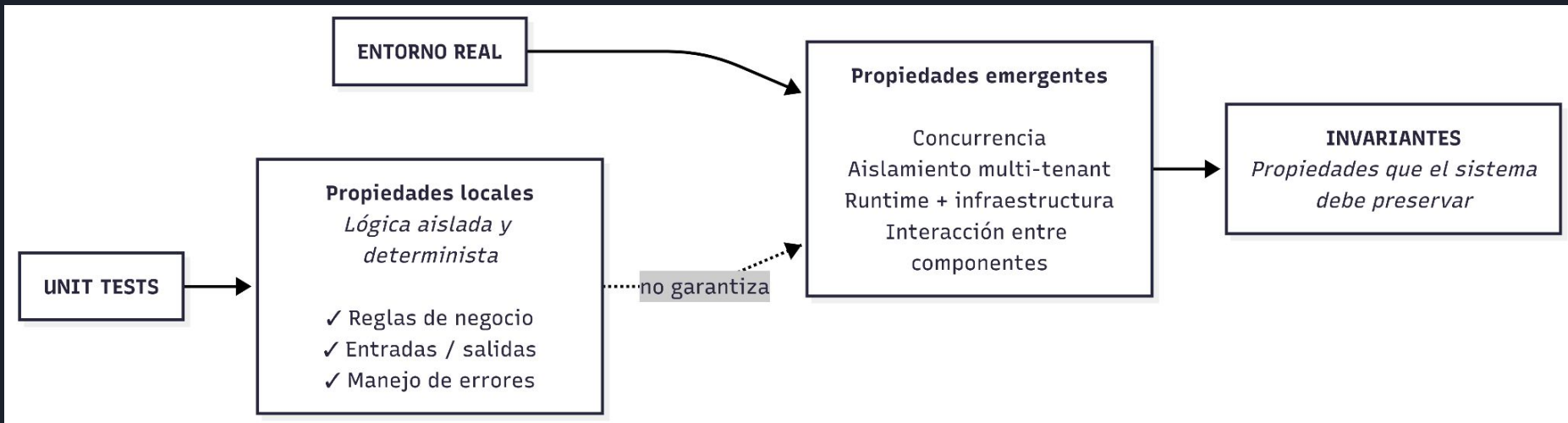


El Alcance de la Validación Local

¿Qué demuestra realmente un unit test?



Los unit tests verifican propiedades locales.
No demuestran propiedades del sistema completo.



Una función puede comportarse correctamente de forma aislada y, aun así, el sistema puede violar una propiedad arquitectónica.

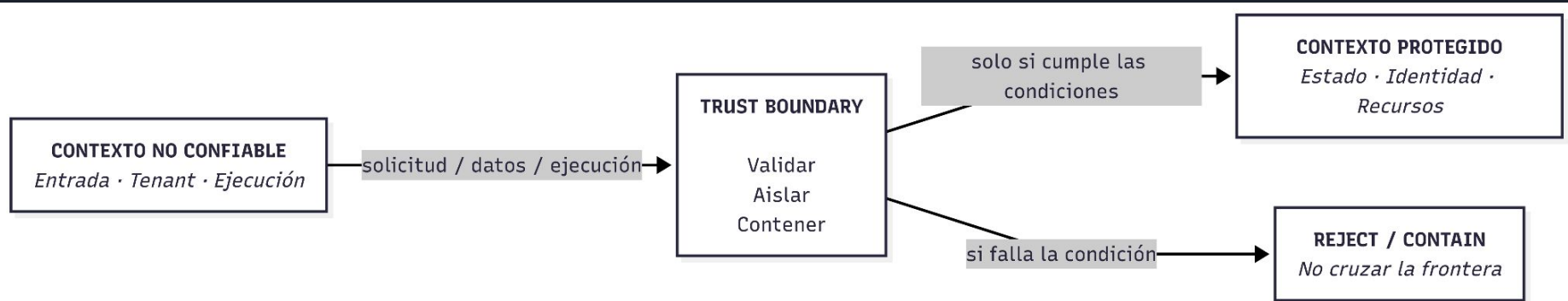
Marco Conceptual: Límites de Confianza

De la suposición a la evidencia

Un Trust Boundary es una frontera explícita donde cambia lo que el sistema está dispuesto a asumir.

Toda frontera debe responder tres preguntas:

1. ¿Qué protegemos?
2. ¿Qué permitimos cruzar?
3. ¿Qué ocurre si la condición no se cumple?



Diseño Basado en Invariantes

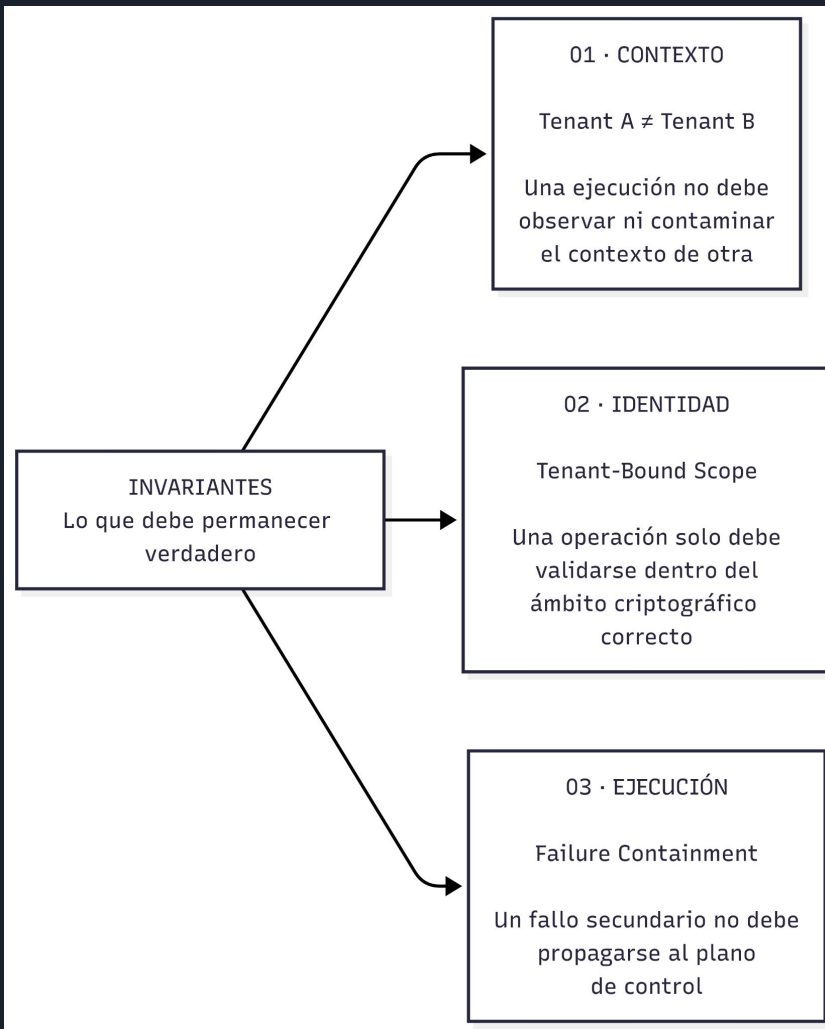
Primero define lo que no puede romperse

Una invariante es una propiedad que el sistema debe preservar durante su ejecución, bajo las condiciones definidas por el diseño.

Estas reglas describen propiedades que el sistema debe preservar, no los pasos concretos con los que las implementa.

Tres propiedades. Tres fronteras. Un mismo objetivo:

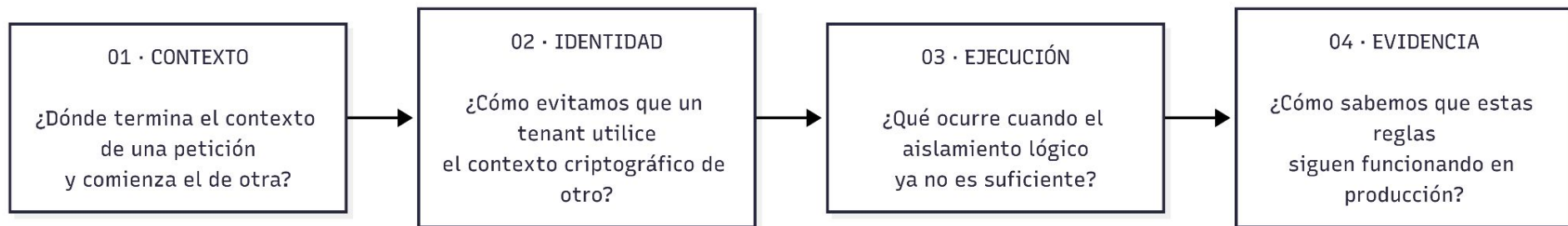
Limitar qué puede afectar un fallo o una ejecución fuera de lo esperado.





Las preguntas que guiaron el diseño

Durante el diseño de la arquitectura, tuvimos que traducir esas invariantes en preguntas concretas sobre nuestras fronteras.



Cada pregunta identifica una propiedad que debemos proteger y un límite que debemos definir.

Las siguientes decisiones de arquitectura son las respuestas a cada una de ellas.

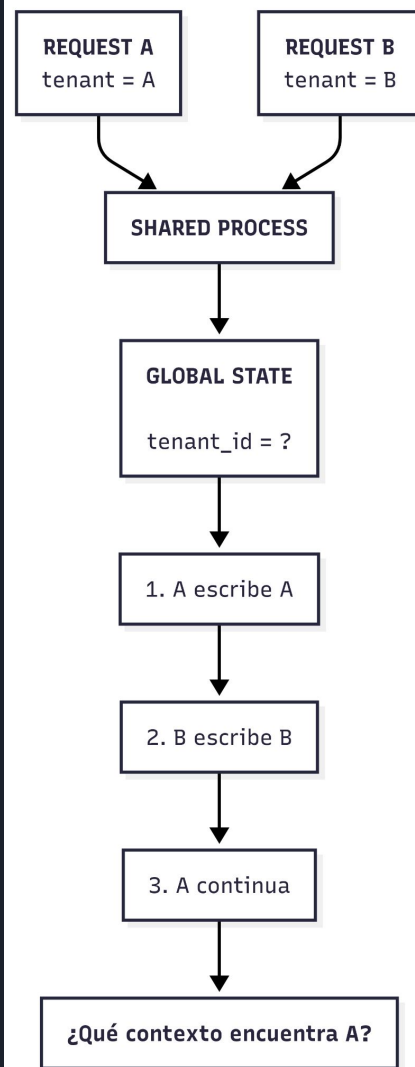
Desafío 1: Concurrencia en runtimes asíncronos

Desafío 1 — ¿Dónde vive el contexto?

El proceso puede ser compartido. El contexto lógico no debería serlo.

El problema: En flujos asíncronos (ej. asyncio), múltiples peticiones se intercalan dentro del mismo proceso.

El riesgo de diseño: Utilizar estado global compartido provoca que las peticiones sobrescriban o contaminen el contexto de otros inquilinos (tenants).



Respuesta 1: Aislamiento de contexto lógico

RESPUESTA 1: Aislar el contexto lógico

Invariante: Cada ejecución mantiene su contexto lógico aislado de otras ejecuciones

Mecanismo: ContextVar + establecimiento y restauración explícita del contexto.

Límites de la solución: ContextVar != Memory Sandbox & Process Isolation (SO)

```
from contextvars import ContextVar

tenant_id: ContextVar[str] = ContextVar("tenant_id")

async def handle(request):
    token = tenant_id.set(request.tenant_id)
    try:
        await do_work()
    finally:
        tenant_id.reset(token)
```

Context A
tenant_id = A

Aislamiento lógico por
corrutina

Context B
tenant_id = B

CONTEXTVAR (Aislamiento)

Request A
Context A

Request B
Context B

tenant_id = A

tenant_id = B

ESTADO GLOBAL (Contaminación)

Request A
tenant = A

Request B
tenant = B

tenant_id (Global)

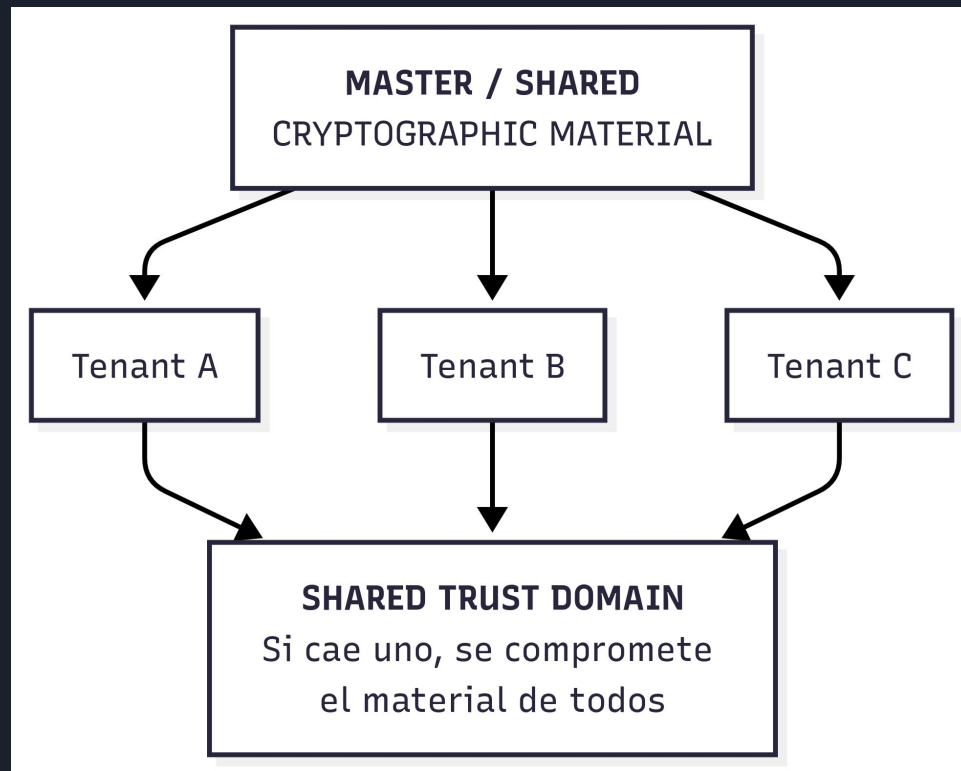
Desafío 2: ¿A qué tenant pertenece esta operación?

El Riesgo de Diseño:

Un dominio criptográfico compartido o demasiado amplio incrementa drásticamente el blast radius (radio de impacto).

La Pregunta:

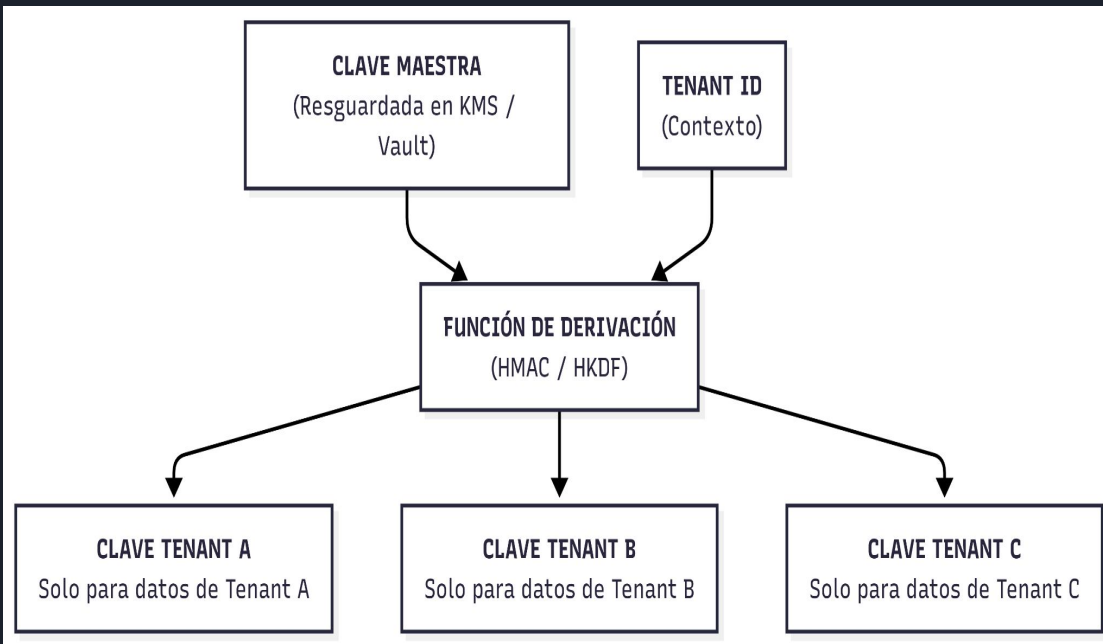
¿Cómo ligamos criptográficamente el material de firma o cifrado al ámbito exclusivo de cada tenant?



Respuesta 2: Aislamiento criptográfico derivado por tenant

Derivamos claves exclusivas por tenant para que el compromiso de un inquilino no exponga a los demás.

Derivación basada
en HMAC
(simplificada)



```
import hashlib
import hmac

def derive_tenant_key(
    master_key: bytes,
    tenant_id: str
) -> bytes:
    """
    Deriva una clave específica por tenant mediante
    HMAC-SHA256.
    """
    info = f"tenant-scope:{tenant_id}".encode("utf-8")

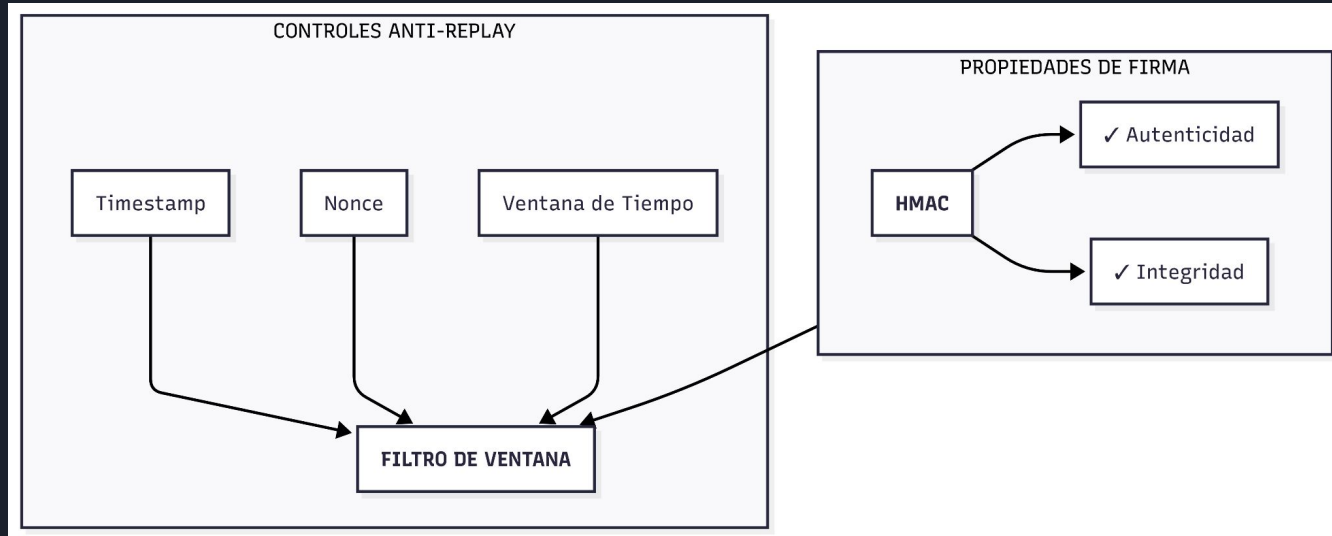
    derived_key = hmac.new(
        master_key,
        info,
        hashlib.sha256
    ).digest()

    return derived_key
```

Respuesta 2: Aislamiento criptográfico derivado por tenant

HMAC permite verificar autenticidad e integridad respecto de la clave secreta asociada al contexto esperado.
El Timestamp + Ventana de tiempo asegura que el mensaje no sea viejo.

Nonce + registro de nonces aceptados → detección de replay dentro de la ventana definida.



Invariante & Advertencia:

HMAC no evita la retransmisión (replay) por sí solo. Se requiere HMAC (quién eres e integridad) + Timestamp y Nonce (cuándo y cuántas veces) para blindar la frontera de identidad.

Desafío 3: ¿Qué pasa cuando el runtime ya no es suficiente?

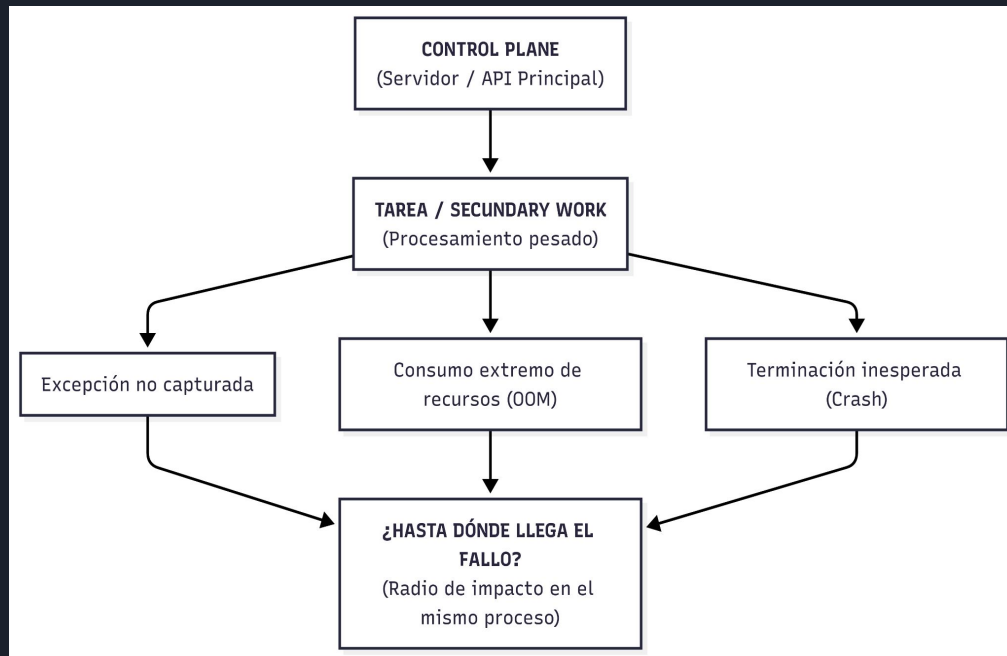
Frontera Lógica vs. Física: ContextVar aísla datos de corrutinas, no aísla recursos ni memoria física del sistema operativo.

El Problema del Noisy Neighbor: Si el trabajo pesado de un tenant agota la memoria (OOM – Out of Memory) o satura la CPU, puede afectar la disponibilidad de todos los demás tenants en el mismo proceso.

Propagación del Crash: Un fallo fatal en una tarea secundaria derriba el Control Plane.

El Riesgo de Diseño: ¿Hasta dónde llega el fallo?

Sin una frontera de proceso, una falla puede afectar a otros componentes que compartan ese proceso.

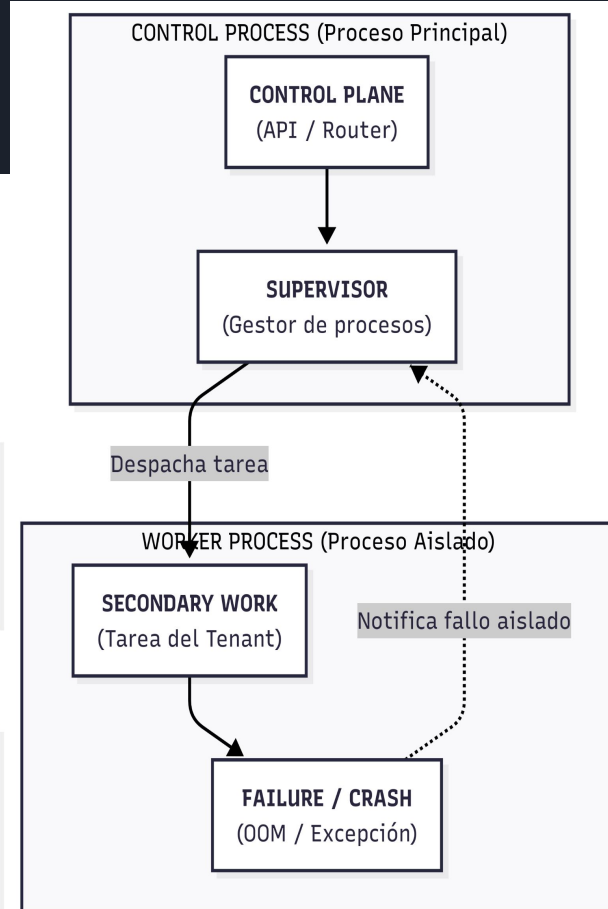


Respuesta 3: Fronteras de Proceso (Process Boundaries)

1. WORKER FAILS
(El proceso hijo muere)

2. CONTROL PROCESS
(Permanece intacto y responde)

3. CONTAIN & RESTART
(Aísla el fallo y reinicia el worker)



Separar los ciclos de vida de ejecución limita el blast radius de una falla dentro del proceso secundario. Un worker separado no significa aislamiento absoluto.

Aislamiento de Memoria y Contención de Recursos: El trabajo pesado o no confiable corre en un proceso (o contenedor) independiente con espacio de direcciones propio y límites explícitos de CPU, memoria, concurrencia y/o throughput según la infraestructura.

Resiliencia del Control Plane: Si el worker muere por falta de memoria (OOM) o un crash nativo, la falla queda contenida en el proceso secundario; el proceso principal puede detectar su terminación mediante el mecanismo de supervisión y decidir cómo recuperarse.

Contención del Noisy Neighbor: Un tenant queda limitado al ámbito de ejecución y recursos asignados a su worker, sujeto a los límites y dependencias compartidas de la infraestructura.



Despliegue Real: Azure Functions

Las invariantes tienen que sobrevivir fuera de mi laptop

Un diseño no está validado si sus garantías de aislamiento, contexto e identidad se rompen al desplegar en un entorno Serverless.

Aislamiento de contexto: Verificación de que nuestra aplicación establezca y restaure explícitamente el contexto por ejecución, sin contaminación entre invocaciones concurrentes bajo las condiciones evaluadas.

Gestión de Material Criptográfico: Integración con servicios de llaves (Key Vault) para la derivación de material por tenant sin exponer secrets en variables de entorno fijas.

Límites de Ejecución Efímera: Una nueva invocación no debe observar valores de contexto pertenecientes a una invocación anterior. Validamos explícitamente esta propiedad bajo reutilización de procesos y ejecuciones concurrentes.



Regla de Validación: El entorno real de ejecución es parte fundamental de la validación arquitectónica. Si la invariante falla bajo escala en la nube, la invariante nunca existió.

De Arquitectura a Evidencia

Una invariante teórica no es evidencia de seguridad.

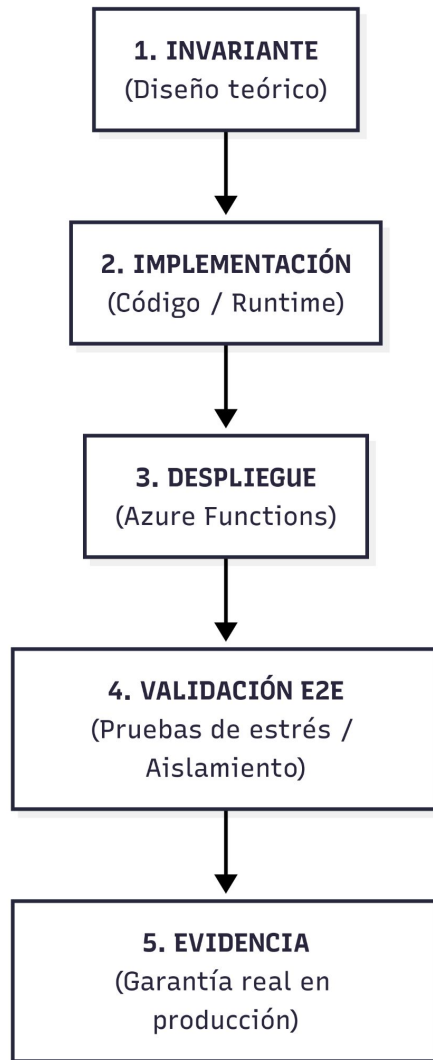
Diseñar no es Demostrar: Diseñar una frontera de aislamiento (ContextVar, derivación basada en HMAC, Process Boundaries) no demuestra que funcione bajo condiciones reales de ejecución.

Comportamiento en Entornos Reales: El runtime asíncrono, los límites de memoria y la escala en la nube en Azure Functions son quienes realmente ponen a prueba las fronteras.

Ingeniería Basada en Evidencia: La seguridad y el aislamiento multitenant no se asumen; se miden, se prueban de extremo a extremo (E2E) y se verifican continuamente.

Regla de Cierre: Diseñar una frontera no demuestra que funciona.

Hay que comprobar y medir su comportamiento en el entorno real.





Preguntas incómodas

Una frontera no elimina el riesgo. Define cómo lo contenemos.

La pregunta no es “¿puede fallar?”

La pregunta es “¿qué puede afectar cuando falle?”

NO BUSCAMOS
aislamiento absoluto



BUSCAMOS
fronteras explícitas



Y MEDIMOS
qué ocurre cuando se cruzan

01 • ¿Y SI CONTEXTVAR NO ES SUFICIENTE?

ContextVar aísla contexto lógico.

¿Qué ocurre cuando necesitamos una frontera de ejecución más fuerte?

02 • ¿Y SI EL SECRETO MAESTRO SE COMPROMETE?

La derivación por tenant acota el ámbito criptográfico.
¿Qué queda fuera de esa frontera?

03 • ¿Y SI FALLA EL PROCESO SECUNDARIO?

Separar procesos reduce el blast radius.
¿Qué ocurre si el host, runtime o infraestructura también falla?



Impact

The goal isn't to prevent every failure.
The goal is to control its blast radius.

Prevent what you can. Contain what
you can't.



Evidence

¿Y cómo sabemos que funciona?

Porque no las probamos únicamente contra mocks o funciones aisladas.

Apider ya está desplegado sobre Azure Functions, y el SDK que estoy utilizando aquí hace las peticiones contra ese deployment real.

Sobre ese entorno ejecutamos la suite completa de integración: 61 checks que atraviesan servicios externos, Webhooks, MCP, errores, concurrencia y diferentes escenarios de ejecución.

61 de 61 pasaron.



Cierre

Diseña el fallo antes de que ocurra.

DEFINE THE BOUNDARY.
CHOOSE THE MECHANISM.
VERIFY THE RESULT.

Jorge de la Flor | Software & Cyber-Physical
Systems Developer

CS Week Perú 2026

